

Aufgabe 3: Lieferkette

Teilnahme-ID: 79083

Bearbeiter dieser Aufgabe:

Abd Alrahman Alkam

13. April 2026

Inhaltsverzeichnis

1 Lösungsidee	3
1.1 Problemstellung	3
1.2 Graphentheoretische Modellierung	3
1.3 Zerlegung in Teilprobleme	4
1.3.1 Das Scheitern von Greedy-Ansätzen	4
1.3.2 Das Scheitern von Backtracking und Simulation	4
1.3.3 Die Wahl der Dynamischen Programmierung	5
1.4 Phase 1: Vorberechnung im ungestörten Graphen	5
1.4.1 Iterativer Dijkstra-Algorithmus	5
1.4.2 Konstruktion einer Matrix für effizientes Path-Routing	5
1.4.3 Tie-Breaking zur Determinismus-Garantie	5
1.5 Phase 2: Rückwärts-DP (Die optimale Restroute)	5
1.5.1 Prinzip der Optimalitätsgleichung im Schichtenmodell	5
1.5.2 Bottom-Up Aufbau ausgehend vom Zielknoten T	5
1.5.3 Speicherung von Pfadhistorien für den ungestörten Fall	5
1.6 Phase 3: Streik-DP	6
1.6.1 Identifikation und Evaluierung valider Ausweichwerke	6
1.6.2 Verhinderung von Selbst-Auswahl und zyklischen Abhängigkeiten	6
1.6.3 Worst-Case-Abschätzung	6
1.7 Phase 4: Vorwärts-DP (Konstruktion des Master-Plans)	6
1.7.1 Akkumulation sicherer Ankunftszeiten (MinArrival)	6
1.7.2 Pruning durch die Frist L	6
1.7.3 Zielprüfung, Synthese und Identifikation des globalen Optimums	6
1.8 Theoretische Grenzen und unlösbare Fälle	7
2 Umsetzung	8
2.1 Datenstrukturen und Abstraktion	8
2.1.1 Globales ID Mapping mittels direktem Array-Zugriff	8
2.1.2 Speichereffiziente Verwaltung der Kanten in einer Adjazenzliste	8
2.2 Architektur der DP-Tabellen	8
2.2.1 Zustandsraum und Dimensionierung ($N \times (m + 1)$ Matrizen)	8
2.2.2 Vermeidung von Rekursionstiefen durch iterative Array-Ansätze	8
2.3 Zurückverfolgung der Route	8
2.3.1 Aufrollen des Graphen über die PrevMain-Matrix	9
3 Laufzeit- und Speicherplatzanalyse	10
3.1 Laufzeitanalyse	10
3.1.1 Deterministische Vorberechnungen: Dijkstra in $\mathcal{O}(N \cdot E \log N)$	10
3.1.2 Verschachtelte Iterationen der DP-Phasen in $\mathcal{O}(m \cdot W_{\max}^2)$	10
3.1.3 Worst-Case	10
3.1.4 Average-Case	10
3.2 Speicherplatzanalyse	11

3.2.1	Dominanz der quadratischen $N \times N$ Distanzmatrizen	11
3.2.2	Cache-Lokalität durch sequentielle Vektoren-Speicherung	11
4	Erweiterungen	12
4.1	Mehrere Streiks	12
4.1.1	Vom singulären zum absoluten Worst-Case	12
4.1.2	Erweiterung des Zustandsraums und Bitmasking	12
4.1.3	Die Min-Max-Optimalitätsgleichung (Bellman)	12
4.1.4	Umsetzung (Top-Down DP)	12
4.1.5	Laufzeit- und Speicherkomplexität	13
4.1.6	Ein- und Ausgabeformat	13
4.2	Stochastische Dynamische Programmierung	15
4.2.1	Vom deterministischen Worst-Case zum Wahrscheinlichkeitsmodell	15
4.2.2	Modell A: Minimierung der erwarteten Gesamtdauer	15
4.2.2.1	Herleitung der Zustandsübergänge (Bellman-Gleichung)	15
4.2.2.2	Bewertung von Modell A	16
4.2.3	Modell B: Maximierung der Fristeinhaltung (Erfolgswahrscheinlichkeit)	16
4.2.4	Herleitung der Zustandsübergänge (Gesetz der totalen Wahrscheinlichkeit)	17
4.2.5	Begründung für die Wahl von Modell B	17
4.2.6	Generierung der Streikwahrscheinlichkeiten	18
4.2.7	Umsetzung	18
4.2.7.1	Zustandsraum und Datenstrukturen	18
4.2.7.1.1	Effiziente Mengenverwaltung via Bitmasking	18
4.2.7.2	Caching der Erfolgswahrscheinlichkeiten	18
4.2.7.3	Extraktion und Konstruktion der Policy	19
4.2.8	Laufzeit- und Speicherplatzanalyse der stochastischen Erweiterung	19
4.2.8.1	Laufzeitanalyse	19
4.2.8.2	Speicherplatzanalyse	20
4.2.8.3	Interpretation einer Ausgabe (Beispiel: lieferung03.txt)	20
4.2.9	Theoretische Robustheit gegenüber mehreren Ausfällen	21
4.2.10	Fazit zur Erweiterung	21
5	Werkzeuge	22
5.1	Entwicklungsumgebung	22
5.2	Einsatz künstlicher Intelligenz (Gemini Pro 3.1)	22
6	Beispiele	23
6.1	Pflichtbeispiele	23
6.1.1	lieferung00.txt	23
6.1.2	lieferung01.txt	23
6.1.3	lieferung02.txt	23
6.1.4	lieferung03.txt	24
6.1.5	lieferung04.txt	24
6.1.6	lieferung05.txt	24
6.1.7	lieferung06.txt	24
6.1.8	lieferung07.txt	24
6.1.9	lieferung08.txt	24
6.1.10	lieferung09.txt	24
6.1.11	lieferung10.txt	25
6.2	Eigene Beispiele	25
6.2.1	greedy_falle.txt	25
6.2.2	speicher_falle.txt	26
6.2.3	alternativen_explosion.txt (MDP-Erweiterung)	26
7	Quellcode	27
8	Literaturverzeichnis	33

1 Lösungsidee

1.1 Problemstellung

Es sind $K = \{1, 2, \dots, m\}$ Arbeitsschritte gegeben, welche sequentiell abgearbeitet werden müssen. Für jeden Schritt $k \in K$ ist eine diskrete Menge fähiger Werke W_k definiert. Außerdem existieren ein Startpunkt S , ein Zielpunkt T sowie eine zeitliche Obergrenze $L \in \mathbb{R}^+$.

Eine valide Normalroute entspricht demnach einem Plan $P_{\text{norm}} = (w_1, w_2, \dots, w_m)$, bei dem die Bedingung $w_k \in W_k$ für alle $k \in K$ erfüllt ist. Die Gesamtdauer solch einer fehlerfreien Route ergibt sich additiv aus den Distanzen zwischen den sequentiellen Stationen:

$$\text{dur}(P_{\text{norm}}) = d(S, w_1) + \sum_{k=1}^{m-1} d(w_k, w_{k+1}) + d(w_m, T) \quad (1)$$

Singuläre Ausfallbedingungen (Streiks) heben dabei die Komplexität des Problems. Für exakt ein beliebiges $k \in K$ kann das gewählte Werk w_k zum Zeitpunkt der Ankunft ausfallen. In diesem Fall muss ein Ausweichwerk $w'_k \in W_k \setminus \{w_k\}$ gewählt werden. Ab diesem w'_k muss eine neue Restroute $(w'_{k+1}, \dots, w'_m)$ für die verbleibenden Schritte festgelegt werden. Für jeden Arbeitsschritt k existiert somit eine Alternativroute P_{alt}^k mit der Gesamtdauer:

$$\text{dur}(P_{\text{alt}}^k) = d(S, w_1) + \sum_{i=1}^{k-1} d(w_i, w_{i+1}) + d(w_k, w'_k) + \sum_{j=k}^{m-1} d(w'_j, w'_{j+1}) + d(w'_m, T) \quad (2)$$

Das Ziel ist es, einen Normalplan P_{norm} und die m zugehörigen Ausweichpläne P_{alt}^k so zu bestimmen, dass die Frist-Bedingung für alle Szenarien gilt:

$$\max\left(\text{dur}(P_{\text{norm}}), \max_{k \in K}(\text{dur}(P_{\text{norm}}|_k) + \text{dur}(P_{\text{alt}}^k))\right) \leq L \quad (3)$$

Dabei beschreibt $P_{\text{norm}}|_k = (w_1, w_2, \dots, w_k)$ den Teil des Normalplans bis zum betroffenen Werk w_k . Die Funktion $\text{dur}(P_{\text{norm}}|_k)$ gibt die bis zur Ankunft am Werk w_k bereits verstrichene Zeit an, während $\text{dur}(P_{\text{alt}}^k)$ die Dauer der ab diesem Punkt notwendigen Umleitungsroute (inklusive der Restroute zum Ziel T) beschreibt.

Ist diese Menge an gültigen Plänen nicht leer, muss aus ihr eine Kombination aus Normal- und Ausweichplänen erstellt werden, die das Maximum aller potenziellen Laufzeiten (den globalen Worst-Case) minimiert.

1.2 Graphentheoretische Modellierung

Um die Distanzfunktion $d(u, v)$ für die Auswertung bereitzustellen, wird die Infrastruktur als ungerichteter, gewichteter Graph $G = (V, E, t)$ modelliert. Die Knotenmenge $V = \{v_1, v_2, \dots, v_n\}$ beinhaltet alle Orte (S, T , alle Werke $w \in W_k$). Die Kantenmenge E repräsentiert das befahrbare Straßennetz mit der Gewichtungsfunktion $t(u, v)$ für die Fahrdauer. Die Distanz $d(u, v)$ entspricht dann der Summe der Kantengewichte auf dem kürzesten Pfad zwischen u und v im Graphen G . Dadurch, dass die Arbeitsschritte sequentiell abgearbeitet werden müssen, transformiert sich das Problem in ein Schichtenmodell, bei dem sich der LKW gerichtet von einer Teilmenge W_k zur nächsten Teilmenge W_{k+1} bewegt (siehe Abbildung 1).

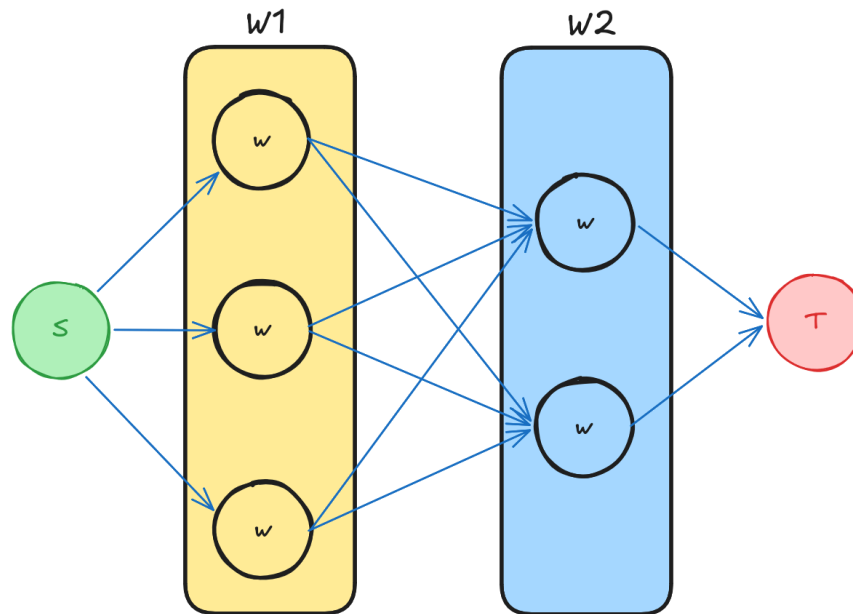


Abbildung 1: Abstraktion des Straßennetzes als gerichteter azyklischer Graph (DAG) von Schicht zu Schicht

1.3 Zerlegung in Teilprobleme

1.3.1 Das Scheitern von Greedy-Ansätzen

Ein Greedy-Ansatz würde versuchen, von S ausgehend stets das zeitlich nächste Werk für jeden Schritt anzufahren. Diese Methode scheitert jedoch an der Streik-Bedingung. Angenommen, das für Schritt k zeitlich nächste Werk w_k liegt isoliert auf einer Halbinsel. Die Fahrt dorthin ist kurz. Fällt w_k jedoch durch einen Streik aus, wäre die Distanz zum nächsten Ausweichwerk w'_k so groß, dass die globale Frist L zwingend gerissen wird. Der Greedy-Ansatz bemerkt diese Sackgasse erst, wenn es zu spät ist. Für korrekte Entscheidungen im späteren Verlauf ist zwingend das Wissen über die zukünftige Resttopologie erforderlich.

1.3.2 Das Scheitern von Backtracking und Simulation

Neben dem Greedy Ansatz wurden weitere Standardverfahren geprüft, die sich für die strikten Anforderungen der Aufgabenstellung jedoch als unzureichend erwiesen. Eine intuitive Methode ist das Backtracking, bei der rekursiv jeder mögliche Pfad vom Start S bis zum Ziel T generiert und anschließend simuliert wird, ob die Route jedem möglichen Streik standhält. Dieser Ansatz leidet jedoch unter einer massiven kombinatorischen Fülle. Da Backtracking die Eigenschaft der optimalen Teilstruktur ignoriert, wird die gesamte Zukunft ab einem Werk u mehrfach neu berechnet, wenn dieses über verschiedene Wege erreicht wird. Bei m Arbeitsschritten und $W_{\max}$ fähigen Werken pro Schritt wächst der Suchbaum exponentiell in $\mathcal{O}(W_{\max}^m)$, was die Lösungsfindung für realistische Graphen unmöglich macht.

Ebenso scheitert ein stochastischer Ansatz via Monte-Carlo-Simulation. Anstatt das Netzwerk exakt durchzurechnen, könnten hierbei Millionen zufälliger Routen generiert und durch simulierte Streiks getestet werden, um statistisch den sichersten Weg zu ermitteln. Eine solche Simulation nähert sich der Wahrheit jedoch nur an und bietet keine Worst-Case-Garantie. Selbst wenn eine Route Millionen simulierte Streik-Szenarien fristgerecht übersteht, schließt dies die Existenz einer hochspezifischen, fatalen Streik-Kombination, welche die Frist L reißt, nicht absolut aus. Da ein Weltraumstart bei Verfehlen der Deadline scheitert, ist eine rein probabilistische Sicherheit nicht ausreichend.

1.3.3 Die Wahl der Dynamischen Programmierung

Aufgrund der Eigenschaft der optimalen Teilstruktur (ein Teilpfad eines kürzesten Pfades ist wieder ein kürzester Pfad) und sich stark überlappenden Teilprobleme (ein Werk kann über diverse Permutationen erreicht werden, die Restroute bleibt aber topologisch identisch), bietet sich die Dynamische Programmierung (DP) als Lösungsart gut an. Das Problem lässt sich in vier Phasen zerlegen: Vorberechnung der Basisdistanzen, Rückwärts-Evaluierung des ungestörten Graphen, Isolierung der lokalen Streikkosten und schließlich die Vorwärts-Zusammensetzung unter Pruning-Bedingungen.

1.4 Phase 1: Vorberechnung im ungestörten Graphen

Bevor dynamische Zustandsübergänge für Werke evaluiert werden können, muss die Distanzfunktion $d(u, v)$ bei allen Knotenpaaren angewandt werden.

1.4.1 Iterativer Dijkstra-Algorithmus

Das Problem entspricht dem All-Pairs Shortest Path (APSP) Problem. Da Fahrtauern niemals negativ sein können ($t(u, v) \geq 0$), wurde der Dijkstra-Algorithmus als Verfahren gewählt [1]. Er wird in einer iterativen Schleife ausgeführt, worin jeder Knoten aus V nacheinander als Startknoten fungiert. Somit wird in jedem Schritt der Knoten mit der aktuell minimalsten Distanz entnommen und gespeichert.

1.4.2 Konstruktion einer Matrix für effizientes Path-Routing

Die reine Kenntnis über die Dauer eines kürzesten Pfades reicht nicht aus, um später die geforderten Routenverläufe auszugeben. Parallel zur Distanzmatrix wird eine Routing-Tabelle aufgebaut. Ein benachbarter Knoten v von u liegt exakt dann auf einem mathematisch optimalen Pfad zu dst , wenn die Kantensumme der folgenden Bedingung entspricht: $d(dst, v) + t(u, v) = d(dst, u)$. Diese Matrix erlaubt später eine effiziente Backtracking-freie Rekonstruktion jedes Pfadstücks im Straßennetz, um eine finale Route zu erhalten.

1.4.3 Tie-Breaking zur Determinismus-Garantie

In dichten Straßennetzen existieren häufig mehrere Pfade oder alternative Werke mit exakt der gleichen Fahrtdauer. Um bei solchen Tie-Breaker-Szenarien eine deterministische Route gewährleisten zu können, nutzt der Algorithmus eine numerische Ordnung. Sobald zwei gleich lange Wege zu einem Ziel führen oder zwei Werke dieselben Kosten verursachen, entscheidet der Operator der Integer-IDs ($u < v$).

1.5 Phase 2: Rückwärts-DP (Die optimale Restroute)

Um im Vorwärtsverlauf die Überlebensfähigkeit eines Werkes beurteilen zu können, muss man genau wissen, wie viel Zeit die optimale Vollendung der Route ab diesem Knoten benötigt.

1.5.1 Prinzip der Optimalitätsgleichung im Schichtenmodell

Die Matrix $\text{MinDist}[u][k]$ wird definiert. Sie speichert die minimale verbleibende Reisezeit von einem Werk u , an dem gerade Schritt k erfolgreich abgeschlossen wurde, bis zum finalen Zielknoten T . In dieser Phase wird idealisiert davon ausgegangen, dass keine Streiks auf der Route auftreten werden.

1.5.2 Bottom-Up Aufbau ausgehend vom Zielknoten T

Die Berechnung erfolgt als Rückwärts-Induktion basierend auf dem Bellmanschen Optimalitätsprinzip [2]. Der Basisfall initialisiert den letzten Arbeitsschritt m : $\text{MinDist}[u][m] = d(u, T)$ für alle $u \in W_m$. Anschließend verbreitet die DP iterativ rückwärts durch die Schichten $k = m - 1$ bis 1. Für jedes Werk $u \in W_k$ werden alle potentiellen Folge-Werke $v \in W_{k+1}$ evaluiert: $\text{MinDist}[u][k] = \min_{v \in W_{k+1}} (d(u, v) + \text{MinDist}[v][k + 1])$

1.5.3 Speicherung von Pfadhistorien für den ungestörten Fall

Jeder Zustandsübergang, der ein neues Minimum für $\text{MinDist}[u][k]$ setzt, überschreibt simultan die Hilfsmatrix $\text{NextNormal}[u][k] = v$. Diese Matrix deklariert, welches Werk, für eine optimale ungestörte Route, als Nächstes angefahren werden muss.

1.6 Phase 3: Streik-DP

Die dritte Phase isoliert die Kosten eines potenziellen Streiks und bewertet das Risiko jedes einzelnen Werks.

1.6.1 Identifikation und Evaluierung valider Ausweichwerke

Tritt an Werk $u \in W_k$ ein Streik auf, muss ein Ausweichwerk gesucht werden.

Die Matrix $\text{BestAltDist}[u][k]$ definiert die minimal notwendige Zeit, um von diesem geschlossenen Tor u den aktuellen Arbeitsschritt k an einem anderen Ort v nachzuholen und von dort aus die Route bis T zu vollenden. $\text{BestAltDist}[u][k] = \min_{v \in W_k \setminus \{u\}} (d(u, v) + \text{MinDist}[v][k])$.

1.6.2 Verhinderung von Selbst-Auswahl und zyklischen Abhängigkeiten

Die Bedingung $v \neq u$ ist von absoluter Notwendigkeit. Sie verhindert eine zyklische Logik (das Werk weicht auf sich selbst aus) und reflektiert die Realität, dass das bestreikte Werk für diesen LKW unbrauchbar geworden ist.

1.6.3 Worst-Case-Abschätzung

Der Wert in $\text{BestAltDist}[u][k]$ repräsentiert die lokalen Strafkosten. Ein Werk u kann in der Normalroute (MinDist) hervorragend abschneiden, aber in BestAltDist einen extrem schlechten Wert aufweisen, wenn es räumlich von allen anderen W_k -Werken isoliert ist. Diese Entkopplung in zwei getrennte Bewertungsmatrizen ist der Schlüssel zur exakten Bewertung der Krisensicherheit.

1.7 Phase 4: Vorwärts-DP (Konstruktion des Master-Plans)

Mit dem neuen Wissen der Rückwärts-Phasen kann nun der eigentliche Master-Plan von S in Richtung T aufgebaut werden.

1.7.1 Akkumulation sicherer Ankunftszeiten (MinArrival)

Die Matrix $\text{MinArrival}[u][k]$ speichert die zeitlich früheste, ungestörte Ankunftszeit am Werk u für den Schritt k , angesammelt ab dem Startpunkt S . Im ersten Schritt ($k = 1$) ist dies die Fahrtdauer $d(S, u)$. Für spätere Schritte ($k \geq 2$) ergibt sich der Kandidat t durch die Ankunftszeit am Vorgänger $v \in W_{k-1}$ zuzüglich der Transferzeit $d(v, u)$.

1.7.2 Pruning durch die Frist L

Bevor ein Wert t als gültiger Zustand in $\text{MinArrival}[u][k]$ gespeichert wird, muss eine zweifache Überlebensbedingung geprüft werden (Pruning):

1. $t + \text{MinDist}[u][k] \leq L$ (Der LKW erreicht T im Normalfall fristgerecht).
2. $t + \text{BestAltDist}[u][k] \leq L$ (Der LKW erreicht T fristgerecht, selbst wenn u in diesem Moment streikt).

Wird nur eine dieser beiden Ungleichungen nicht eingehalten, ist der Pfad, der zu u geführt hat, nicht gut genug, weswegen der Zustandsübergang verworfen wird.

1.7.3 Zielprüfung, Synthese und Identifikation des globalen Optimums

Pfade, die bis zum letzten Schritt m überleben und für die $t_{\text{final}} = \text{MinArrival}[u][m] + d(u, T) \leq L$ gilt, sind verifizierte Master-Pläne. Unter allen Werken $u \in W_m$, die das Ziel T erreichen können, wird das gewählt, welches das Basis-Weg-Maximum t_{final} minimiert. Bleibt die Menge der gültigen Werke leer, gibt der Algorithmus „UNMOEGLICH“ aus. Andernfalls wird über die Historien-Matrix PrevMain die Route zurückverfolgt und die berechnete, global optimale Lösung zusammengeführt.

Dass der DP-Algorithmus den optimalen Master-Plan findet, lässt sich durch die Logik der aufeinander aufbauenden Phasen begründen. Es handelt sich hierbei um ein exaktes Verfahren und nicht um eine Schätzung wie bei Heuristiken:

1. **Perfekte Teilrouten:** Die Rückwärts-DP arbeitet sich vom Ziel aus rückwärts nach vorne durch. Da die grundlegenden Fahrzeiten durch den Dijkstra-Algorithmus bereits minimal sind, stellt der Wert in MinDist die absolute Untergrenze für die restliche Fahrt dar. Einen schnelleren Weg gibt es im gesamten Straßennetz nicht. Auch die berechneten Streikkosten (BestAltDist) basieren vollständig auf diesen perfekten Restrouten.

2. **Entscheidungen mit voller Voraussicht:** Wie beim Scheitern des Greedy-Ansatzes gezeigt, tappen einfache Algorithmen oft in Fallen, weil sie zukünftige Sackgassen bei einem Streik nicht rechtzeitig erkennen. Der DP-Ansatz löst dieses Problem, indem die Konstruktion der eigentlichen Vorwärts-Route (Phase 4) erst dann startet, wenn die Kosten für jeden möglichen Streik in der Zukunft bereits berechnet und in den Tabellen gespeichert sind. Der Algorithmus weiß also bei jedem Schritt schon exakt, was im schlimmsten Fall passieren wird.
3. **Systematisches Testen aller Wege:** Der Algorithmus übersieht keine mögliche Routenkombination. Die ineinander verschachtelten Schleifen gehen alle fähigen Werke für jeden Arbeitsschritt systematisch durch. Ein Weg wird nur dann frühzeitig abgebrochen (Pruning), wenn mathematisch sicher feststeht, dass die vorgegebene Zeitfrist L im Normal- oder Streikfall nicht eingehalten wird. Aus den Routen, die das Ziel fristgerecht erreichen, wird am Ende einfach die kürzeste ausgewählt. Da somit alle Wege entweder vollständig geprüft oder logisch sicher ausgeschlossen wurden, ist das Ergebnis garantiert die global beste Lösung.

1.8 Theoretische Grenzen und unlösbare Fälle

Bei der abstrakten Betrachtung des Lieferketten-Problems stellt sich die Frage, unter welchen Bedingungen eine Instanz fundamental unlösbar ist und wo die Grenzen der gewählten Lösung liegen.

Eine Unlösbarkeit liegt immer dann vor, wenn die topologischen Grundvoraussetzungen des Graphen verletzt sind oder die Zeitfrist zu strikt ist. Ist der Graph nicht zusammenhängend, sodass kein durchgehender Pfad vom Startpunkt S über alle Arbeitsschritte bis zum Zielpunkt T existiert, scheitert jede Routenplanung. Ebenso ist das Problem unlösbar, wenn die berechnete absolute Untergrenze der ungestörten Fahrtdauer, also die reine Summe der kürzesten Distanzen zwischen den optimalen Werken, bereits die vorgegebene Frist L überschreitet.

Ein weiterer Fall entsteht durch Single Points of Failure. Besitzt ein Arbeitsschritt k exakt nur ein einziges fähiges Werk ($|W_k| = 1$), ist das Problem unlösbar, sobald das Worst-Case-Szenario einen Streik an exakt diesem Werk annimmt. Da die Aufgabenstellung verlangt, gegen den Ausfall jedes angefahrenen Werks gewappnet zu sein, fehlt hier das physische Ausweichwerk. Aber auch ohne singuläre Werke können isolierte Halbinsel-Lagen (wie im konstruierten Beispiel `greedy_falle.txt` demonstriert) zur Unlösbarkeit führen. Das passiert genau dann, wenn der Streik an einem Werk den LKW zu einem langen Umweg zum nächsten Ausweichwerk zwingt, wodurch die Restzeit über das Limit L hinaus strapaziert wird.

Neben diesen Schranken weist der implementierte Algorithmus spezifische Limitierungen auf. Die gewählte Dynamische Programmierung verlässt sich zur Laufzeitminimierung auf eine $\mathcal{O}(1)$ -Zugriffszeit durch eine vorberechnete All-Pairs-Shortest-Path-Distanzmatrix. Während das für reale, dünn besiedelte Straßennetze skaliert, bildet diese Matrix den Bottleneck für riesige Instanzen. Bei Graphen enormer Anzahl an Kreuzungen würde der quadratische Speicherbedarf von $\mathcal{O}(N^2)$ den verfügbaren Arbeitsspeicher moderner Rechner sprengen. Ebenso erzwingen Cliques ein starkes Ansteigen der Kantenanzahl E , was die Initialphase des Dijkstra-Algorithmus stark ausbremst.

2 Umsetzung

Um die Laufzeit des Programmes zu minimieren, wurde das Problem in C++ implementiert. Die Wahl der Container und Algorithmen fokussiert sich primär auf $\mathcal{O}(1)$ -Zugriffszeiten und hohe Cache-Lokalität.

2.1 Datenstrukturen und Abstraktion

2.1.1 Globales ID Mapping mittels direktem Array-Zugriff

Zur Vereinfachung des Routings wird jedem Ort (inklusive S und T) eine eindeutige, numerische ID für die interne Datenverwaltung zugeteilt. Anstatt einer Suchbaum-Struktur (`std::map`), die beim Aufbau logarithmische Zugriffszeiten erfordert, nutzt die Implementierung die Eigenschaft aus, dass alle Werke bereits als Strings von Integer-Werten vorliegen. Zunächst wird die höchste vorkommende Werksnummer `max_w_id` im Datensatz ermittelt. Daraufhin wird ein Vektor `node_names` der Größe $n = \text{max_w_id} + 2$ deklariert. Der Startpunkt S erhält die ID 0, das Ziel T den letzten Index $n - 1$. Alle numerischen Werke erhalten ihre direkte Integer-Repräsentation als ID. Die Lambda-Funktion `get_id` kapselt diese direkte Konvertierung. Diese Methodik eliminiert den Overhead von Baumtraversierungen beim Kanten-Einlesen vollständig und erlaubt sofortige Array-Zugriffe.

2.1.2 Speichereffiziente Verwaltung der Kanten in einer Adjazenzliste

Da reale Straßennetze typischerweise sehr dünn besetzt sind wäre die Allokation einer $N \times N$ Adjazenzmatrix höchst ineffizient.

Stattdessen wird das Straßennetz als Adjazenzliste `vector<vector<Edge>> adj` abgebildet. Das `struct Edge` speichert dabei lediglich den Zielknoten (`to`) und die Fahrtdauer (`weight`). Da das Graphenmodell ungerichtet ist, wird jede Eingabekante (u, v) symmetrisch sowohl in `adj[u]` als auch in `adj[v]` registriert. Dies hält den initialen Speicherbedarf strikt linear zur Anzahl der Straßen und ermöglicht eine hochgradig effiziente Iteration über die Nachbarn beim Dijkstra-Algorithmus.

2.2 Architektur der DP-Tabellen

Die mathematischen Matrizen der Dynamischen Programmierung müssen effizient im RAM abgebildet werden.

2.2.1 Zustandsraum und Dimensionierung ($N \times (m + 1)$ Matrizen)

Jeder Zustand in den DP-Phasen (mit Ausnahme der Basisdistanzen) ist durch exakt zwei Parameter definiert: Den aktuellen Aufenthaltsort (Werk u) und den aktuell betrachteten Arbeitsschritt (k). Die Tabellen `MinDist`, `NextNormal`, `BestAltDist`, `NextAlt`, `MinArrival` und `PrevMain` werden demnach als zweidimensionale `std::vector`-Konstrukte der Dimension $N \times (m + 1)$ allokiert.

Um Integer Overflows bei der Addition von Fahrtdauern zu vermeiden, werden alle Zeit-Matrizen zwingend mit dem 64-Bit-Datentyp `long long` erstellt. Der Spaltenindex $m + 1$ erlaubt eine 1-basierte Indizierung der Arbeitsschritte, was die kognitive Komplexität bei der Übertragung der mathematischen Indizes in den Programmcode minimiert.

2.2.2 Vermeidung von Rekursionstiefen durch iterative Array-Ansätze

Obwohl Dynamische Programmierung theoretisch elegant über rekursive Funktionen mit Memoization (Top-Down) implementiert werden kann, birgt dies bei tiefen Graphen ein großes Risiko. Wenn die Anzahl der Arbeitsschritte m groß ist, droht bei Rekursion ein Stack Overflow.

Daher wurde eine rein iterative Implementierung (Bottom-Up bzw. Top-Down in linearen Schleifen) gewählt. Die verschachtelten `for`-Schleifen iterieren explizit über die Schichten k . Da `std::vector` in C++ seine Elemente linear am Stück speichert, führt das systematische, zeilenweise Auslesen der Matrizen in den iterativen Schleifen zu einer starken Ausnutzung der CPU-Hardware-Caches. Dieser sogenannte „Cache-Hit“-Vorteil macht die Array-Lösung in der Praxis um ein Vielfaches schneller als objektorientierte Baum-Traversierungen.

2.3 Zurückverfolgung der Route

Die pure Berechnung des Optimums nützt wenig, wenn der Weg dorthin nicht extrahiert werden kann.

2.3.1 Aufrollen des Graphen über die PrevMain-Matrix

Nach Abschluss der Phasen identifiziert der Algorithmus das Zielwerk `best_w_m` (das Werk des letzten Schritts m , welches den Master-Plan global minimiert). Um die Chronologie zu rekonstruieren, wird ein Vektor W der Länge $m + 1$ allokiert. Beginnend am Ende ($W[m] = \text{best_w_m}$) iteriert eine Rückwärtsschleife von $k = m$ bis 2. Das jeweils vorherige Werk durch die Abfrage $W[k - 1] = \text{PrevMain}[W[k]][k]$ ermittelt. Am Ende der Schleife enthält W die chronologisch korrekte Sequenz aller anzufahrenden Werke der Hauptroute.

3 Laufzeit- und Speicherplatzanalyse

Um die Skalierbarkeit des Algorithmus auf große Straßennetze zu bewerten, wird eine asymptotische Komplexitätsanalyse durchgeführt. Hierbei werden folgende Parameter definiert:

- N : Anzahl der eindeutigen Knoten (Orte/Werke) im Graphen.
- E : Anzahl der ungerichteten Straßenverbindungen.
- m : Anzahl der sequentiellen Arbeitsschritte.
- W_k : Die Knoten-Menge aller Werke, die Arbeitsschritt k anbieten.
- $W_{\max} = \max(|W_k|)$: Die Obergrenze der Werksanzahl für einen einzelnen Arbeitsschritt.

3.1 Laufzeitanalyse

Die Gesamtlaufzeit setzt sich additiv aus den Vorbereitungsphasen und den Iterationen der Dynamischen Programmierung zusammen.

3.1.1 Deterministische Vorberechnungen: Dijkstra in $\mathcal{O}(N \cdot E \log N)$

Das All-Pairs Shortest Path Problem (APSP) muss gelöst werden, um die Distanzmatrix zu füllen. Der Dijkstra-Algorithmus wird iterativ auf jeden der N Knoten als Startpunkt angewendet. Die Implementierung nutzt einen Min-Heap (`std::priority_queue`). Das Einfügen und Extrahieren eines Knotens aus dem Heap kostet logarithmische Zeit $\mathcal{O}(\log N)$. Da im schlimmsten Fall jede Kante E einmal geprüft und potenziell ein Update im Heap auslösen kann, benötigt ein einzelner Dijkstra-Durchlauf maximal $\mathcal{O}(E \log N)$ Operationen.

Da dieser Vorgang N -mal wiederholt wird, ergibt sich für die erste Phase eine strikte Laufzeitschranke von $\mathcal{O}(N \cdot E \log N)$. Das anschließende Befüllen der `next_node`-Routing-Matrix iteriert nochmals über alle Kanten für alle Knotenpaare und benötigt damit konstante $\mathcal{O}(N \cdot E)$, was im Grenzwert von dem logarithmischen Term dominiert wird.

3.1.2 Verschachtelte Iterationen der DP-Phasen in $\mathcal{O}(m \cdot W_{\max}^2)$

Die Phasen 2, 3 und 4 (Rückwärts-DP, Streik-DP, Vorwärts-DP) basieren auf der Evaluierung aufeinanderfolgender Arbeitsschritte. In diesen Phasen iterieren verschachtelte Schleifen über die Werke des aktuellen Schritts W_k und vergleichen sie mit den Werken des Folgeschritts W_{k+1} (bzw. bei der Streik-DP mit Ersatzwerken in W_k , oder bei der Vorwärts-DP mit Vorgängern in W_{k-1}).

Die maximale Anzahl an Vergleichen für den Statusübergang zwischen zwei Schichten k und $k+1$ beträgt $|W_k| \cdot |W_{k+1}|$. Da dies für alle m Schritte berechnet werden muss, summiert sich der Aufwand im Maximum auf:

$$\sum_{k=1}^{m-1} |W_k| \cdot |W_{k+1}| \leq m \cdot W_{\max}^2 \quad (4)$$

Da die Anzahl der fähigen Werke pro Schritt niemals die Gesamtzahl aller Knoten N überschreiten kann ($W_{\max} \leq N$), ist dieser dynamische Teilalgorithmus im absoluten Worst-Case durch $\mathcal{O}(m \cdot N^2)$ beschränkt.

3.1.3 Worst-Case

Zusammengefasst lautet die absolute Worst-Case-Laufzeit des gesamten Programms:

$$\mathcal{O}(N \cdot E \log N + m \cdot W_{\max}^2) \quad (5)$$

Sollte der unnatürliche Fall eintreten, dass das gesamte Straßennetz ein vollständig vernetzter Graph (eine Clique) ist, in dem jeder Ort mit jedem anderen Ort direkt verbunden ist, gilt $E = \mathcal{O}(N^2)$. Eingesetzt in die obige Formel würde dies die theoretische Laufzeit auf $\mathcal{O}(N^3 \log N)$ ansteigen lassen.

3.1.4 Average-Case

In der Praxis – und auch bei den realitätsnahen Testfällen – sind Straßenkarten stark ausgedünnt und weisen annähernd planare Eigenschaften auf. Der durchschnittliche Kantengrad eines Knotens (wie viele

Straßen von einer Kreuzung abgehen) ist fast immer eine kleine Konstante (z.B. 3 oder 4). Dadurch wächst E nicht quadratisch, sondern linear mit N , also $E = \mathcal{O}(N)$.

Unter diesen praxisnahen Bedingungen fällt die Laufzeit der APSP-Dijkstra-Phase auf ein äußerst gutes $\mathcal{O}(N^2 \log N)$.

Ein weiterer drastischer Beschleunigungsfaktor ist das algorithmische Pruning (Ausdünnen des Suchraums) in der Phase 4. Zwar liegt die theoretische Schranke der DP bei $\mathcal{O}(m \cdot W_{\max}^2)$, jedoch bricht der Algorithmus durch die `continue`-Anweisung Evaluationen sofort ab, wenn ein Zustand in `MinArrival` als ungültig erkannt wird. Sobald die Vorab-Überprüfung zeigt, dass die Zeitrestriktion $t + \text{BestAltDist}[u][k] > L$ verletzt wird, entfallen weite Teile des nachfolgenden Suchbaums. In der Praxis führt dies dazu, dass die DP-Phasen nur einen sehr geringen Bruchteil der Gesamtlaufzeit beanspruchen und das Programm beinahe direkt nach der Graphen-Vorverarbeitung terminiert.

3.2 Speicherplatzanalyse

Während die Laufzeit stark vom Grad der Vernetzung abhängt, ist der Speicherbedarf (RAM) deterministisch durch die gewählten Vektorenstrukturen vorgegeben.

3.2.1 Dominanz der quadratischen $N \times N$ Distanzmatrizen

Den massivsten Footprint in der Speicherallokation hinterlassen die Resultate aus der Phase 1. Um einen stetigen $\mathcal{O}(1)$ Zugriff auf Fahrtauern und Routing-Pfade während der DP-Phasen zu garantieren, werden zwei zweidimensionale Matrizen angelegt:

- `dist_to`: Speichert die minimalen Reisezeiten (als 8-Byte `long long`).
- `next_node`: Speichert den direkten Nachbarknoten-Index (als 4-Byte `int`).

Beide Matrizen haben die exakte Dimension $N \times N$. Der Speicherbedarf hierfür skaliert deterministisch quadratisch in $\mathcal{O}(N^2)$. Bei einer Knotenanzahl von beispielsweise $N = 2000$ erzeugt die `dist_to`-Matrix 4.000.000 Einträge. Mit 8 Byte pro Element entspricht dies etwa 32 Megabyte Arbeitsspeicher. Diese Größe ist auf jeder modernen Consumer-Hardware vollkommen problemlos speicherbar.

Die eigentlichen DP-Tabellen (wie `MinDist`) fallen mit einer Dimension von $N \times (m + 1)$ speichertechnisch nicht ins Gewicht, da typischerweise $m \ll N$ gilt. Der asymptotische Gesamtspeicherplatz für das Programm ist daher exakt als $\mathcal{O}(N^2 + E + m \cdot N)$ zu beziffern.

3.2.2 Cache-Lokalität durch sequentielle Vektoren-Speicherung

Die Entscheidung, den C++-Container `std::vector` für die Realisierung der $N \times N$ Arrays zu nutzen, wirkt sich sehr positiv auf die reale Hardware-Performance aus. Im Gegensatz zu abstrakten Knoten-Objekten mit verstreuten Pointer-Strukturen, die den Speicher fragmentieren, sichert der Vektor eine streng zusammenhängende Allokation im Arbeitsspeicher.

Wenn der Prozessor in den DP-Schleifen sequentiell über die Werke W_k iteriert, werden ganze Datenblöcke in den schnellen L1/L2-Cache der CPU geladen. Die Hardware-Prefetcher der CPU können lineare Array-Aufrufe perfekt vorhersehen. Dieser unsichtbare Architekturvorteil begründet maßgeblich, warum der Array-fokussierte Algorithmus selbst riesige Datenmengen innerhalb weniger Millisekunden bewältigen kann.

4 Erweiterungen

4.1 Mehrere Streiks

4.1.1 Vom singulären zum absoluten Worst-Case

Der ursprüngliche Basis-Algorithmus garantiert die Fristeinhaltung unter der Voraussetzung, dass im gesamten Prozess maximal ein einziges Werk ausfällt. Um das System gegen schwerwiegendere Ausfallszenarien abzusichern, wird das Modell nun verallgemeinert: Es können s beliebige Werke bestreikt darf.

Das Ziel des Algorithmus ist es, eine Strategie zu finden, welche die garantierte Gesamtfahrzeit t selbst dann minimiert, wenn die s Streiks maximal ungünstig platziert sind. Die einzige logische Restriktion hierbei ist, dass pro Arbeitsschritt mindestens ein fähiges Werk geöffnet bleiben muss, da die Mission andernfalls unlösbar wäre.

4.1.2 Erweiterung des Zustandsraums und Bitmasking

Um das mehrfache Anfahren desselben geschlossenen Werks auszuschließen, reicht die bisherige zweidimensionale Matrizen-Struktur nicht mehr aus. Der Zustandsraum wird zu einem 4-Tupel (x, k, U, s) erweitert:

- x : Der aktuelle Aufenthaltsort (Knoten-ID).
- k : Der aktuell auszuführende Arbeitsschritt.
- $U \subseteq W_k$: Die Menge der für Schritt k noch nicht besuchten (und damit noch potenziell offenen) Werke.
- s : Die noch verbleibende Anzahl an möglichen Streiks des Gegners.

Da die maximale Anzahl fähiger Werke $W_{\max}$ pro Schritt in den Beispielen überschaubar bleibt, wird die Restmenge U als Integer-Bitmaske abgebildet. Jedes Bit repräsentiert dabei die aktuelle Verfügbarkeit eines Werkes.

4.1.3 Die Min-Max-Optimalitätsgleichung (Bellman)

Die Suche nach der minimalen garantierten Restzeit $C(x, k, U, s)$ vom aktuellen Zustand bis zum Ziel T lässt sich als Bellman-Gleichung formulieren. In jedem Zustand wählt der Algorithmus minimierend das nächste bestmögliche Werk $u \in U$. Sobald der LKW ankommt, stellt sich heraus ob im Werk gestreikt wird oder nicht.

$$C(x, k, U, s) = \min_{u \in U} \left(d(x, u) + \max \left(C_{\text{offen}(u)}, C_{\text{geschlossen}(u)} \right) \right) \quad (6)$$

Die Folgekosten der beiden Entscheidungen sind wie folgt:

- **Fall A (Offen):** Der Arbeitsschritt k wird erfolgreich abgeschlossen. Der LKW geht über in die nächste Schicht $k + 1$ mit dem vollen Set an Werken W_{k+1} . Das Streik-Budget s bleibt unverändert:

$$C_{\text{offen}(u)} = C(u, k + 1, W_{k+1}, s) \quad (7)$$

- **Fall B (Geschlossen):** Der Arbeitsschritt k schlägt fehl. Der LKW verbleibt in Schicht k , jedoch wird das bestreikte Werk u aus der Menge der Alternativen U eliminiert. Das Streik-Budget verringert sich um 1:

$$C_{\text{geschlossen}(u)} = \begin{cases} C(u, k, U \setminus \{u\}, s - 1) & \text{falls } s > 0 \\ -\infty & \text{falls } s = 0 \end{cases} \quad (8)$$

Ist das Streikbudget vollständig aufgebraucht ($s = 0$), können keine Streiks mehr ausgeführt werden. Fall B tritt folglich nicht mehr ein, weshalb ausschließlich $C_{\text{offen}(u)}$ evaluiert wird.

Basisfälle: Nach erfolgreichem Abschluss aller m Schritte entspricht die Restzeit der direkten Fahrt zum Ziel: $C(u, m + 1, W_{m+1}, s) = d(u, T)$. Sind im aktuellen Schritt keine wählbaren Alternativen mehr in U vorhanden ($U = \emptyset$), gilt die Mission als gescheitert: $C(x, k, \emptyset, s) = \infty$.

4.1.4 Umsetzung (Top-Down DP)

Die Integration der Bitmaske und der kontinuierlichen Variable s vergrößert den theoretischen Zustandsraum auf $N \times m \times 2^{W_{\max}} \times s$. Da dieser Raum jedoch dünn besetzt ist (viele Bitmask-Kombinationen

und Restzeit-Zustände werden auf kürzesten Pfaden niemals erreicht), wurde der iterative Bottom-Up-Ansatz des Basis-Modells verworfen.

Es wird stattdessen eine rekursive Top-Down Dynamische Programmierung mit Memoization genutzt. Die Datenstruktur `std::map<tuple, long long> memo` hasht jeden besuchten Pfadverlauf und speichert die entsprechende Restzeit. Parallel dazu merkt sich eine zweite Map (`best_action`) das jeweils lokal optimale Werk.

So wird die vollständige Iteration leerer Zustände entgangen und es wird während der Rekursion einen reaktiven Entscheidungsbaum aufgebaut. Nach der Initialisierung lässt sich über die `best_action`-Map eine dynamische Policy auslesen, die nicht nur eine garantierte Hauptroute, sondern auch mehrere Ausweichpfade für bis zu s aufeinanderfolgende Streiks generiert.

4.1.5 Laufzeit- und Speicherkomplexität

Die Integration einer variablen Streikanzahl s verändert die Komplexität stark. Da der Zustandsraum der Top-Down Dynamischen Programmierung jetzt die Dimensionen Aufenthaltsort (maximal $W_{\max}$), Arbeitsschritt (m), verfügbare Werke als Bitmaske ($2^{W_{\max}}$) und das verbleibende Streikbudget (s) umfasst, ergibt sich für die Auswertung der Zustandsübergänge eine Worst-Case-Laufzeit von:

$$\mathcal{O}(m \cdot W_{\max}^2 \cdot 2^{W_{\max}} \cdot s \cdot \log|S|) \quad (9)$$

Hierbei beschreibt $|S|$ die Anzahl der in der Caching-Map `memo` gespeicherten Zustände. Hinzu kommt additiv die Vorberechnung durch den Dijkstra-Algorithmus in $\mathcal{O}(N \cdot E \log N)$.

Der Speicherplatzbedarf wird, neben der quadratischen $N \times N$ Distanzmatrix ($\mathcal{O}(N^2)$), durch die Tiefe der Rekursion und die Größe der Hash-Map dominiert, welche im extremsten Fall $\mathcal{O}(m \cdot W_{\max} \cdot 2^{W_{\max}} \cdot s)$ Einträge speichern muss.

Diese Analyse zeigt einen starken Bottleneck. Während das Modell äußerst flexibel auf multiple Ausfälle reagiert und das Streikbudget s nur linear ins Gewicht fällt, erzwingt die Bitmaske ein exponentielles Wachstum in $2^{W_{\max}}$. Das Verfahren skaliert in der Praxis folglich nur dann effizient, wenn die Topologie stark ausgedünnt ist und die maximale Anzahl an wählbaren Alternativwerken pro Arbeitsschritt eine geringe Grenze nicht überschreitet.

4.1.6 Ein- und Ausgabeformat

Zusätzlich zur Basis-Konfiguration liest das Programm die maximal zulässige Anzahl an Streiks s ein. Die Ausgabe folgt der normalen Struktur, wobei die Zeitangaben nun jedoch das Ergebnis einer komplexeren Min-Max-Kalkulation darstellen.

Beispiel:

Output (zwei_streiks.txt):

```
MOEGLICH
60
40
S [1 1] [4 2] T
60
1 [2 1] [4 2] T
```

Die Zeitangabe von 60 bildet hierbei die globale Worst-Case-Garantie. Selbst bei optimaler Platzierung beider Streiks erreicht die LKW-Fahrerin das Ziel T in spätestens 60 Einheiten. Die Differenz zur Hauptroute (40) zeigt den notwendigen Zeitpuffer für Ausweichmanöver.

Im Gegensatz zum Basis-Algorithmus ist die Fallback-Zeit (z. B. 60 bei einem Streik in Werk 1) keine statische Pfadlänge mehr. Sie ist das Resultat einer vorausschauenden Strategie. Wenn Werk 1 ausfällt,

wird auf Werk 2 ausgewichen. Die Garantie von 60 Einheiten bleibt dabei bestehen, selbst wenn der verbleibende zweite Streik später im Prozess an der ungünstigsten Stelle erfolgt.

Aus Gründen der Übersichtlichkeit visualisiert das Programm nur den primären Ausweichplan. Die berechnete Zeit deckt jedoch intern bereits alle weiteren potenziellen Streik-Kombinationen ab, die das Budget s zulässt. Damit liefert die Erweiterung keine bloßen Pfade, sondern eine vollständige Entscheidungsstrategie durch einen Baum möglicher Ausfälle.

4.2 Stochastische Dynamische Programmierung

4.2.1 Vom deterministischen Worst-Case zum Wahrscheinlichkeitsmodell

Die ursprüngliche Problemstellung geht von einem deterministischen Worst-Case-Szenario aus, bei dem exakt ein Werk bestreikt wird. In der realen Logistik unterliegen solche Ausfälle jedoch meist unabhängigen, stochastischen Gesetzmäßigkeiten. Wir erweitern das Modell daher um probabilistische Ausfallraten.

Jedem Werk $u \in W_k$ wird eine spezifische, im Vorfeld bekannte Streikwahrscheinlichkeit $p_u \in [0, 1]$ zugeordnet. Das Eintreten eines Streiks an Werk u wird als stochastisch unabhängig vom Zustand anderer Werke modelliert. Durch diese Modifikation transformiert sich das Entscheidungsproblem in einen Markov-Entscheidungsprozess (Markov Decision Process, MDP) [3]. Anstatt eine singuläre Min-Max-Bedingung zu verifizieren, erfordert dieses Modell die Optimierung statistischer Größen. Dabei bieten sich zwei grundlegend verschiedene Modelle an.

4.2.2 Modell A: Minimierung der erwarteten Gesamtdauer

Der erste Ansatz zielt darauf ab, den Erwartungswert der benötigten Gesamtfahrzeit zu minimieren. Die Leitfrage lautet hier: *Wie sieht eine Routenplanung aus, die im statistischen Durchschnitt über viele Durchläufe hinweg am wenigsten Zeit in Anspruch nimmt?*

Wir definieren eine dynamische Zustandsfunktion $E(x, k, U)$, welche die minimale zu erwartende Restzeit beschreibt. Der Zustand ist definiert durch:

- x : Den aktuellen Standort des LKWs.
- k : Den aktuell auszuführenden Arbeitsschritt ($1 \leq k \leq m$).
- $U \subseteq W_k$: Die Menge der für den Schritt k noch verfügbaren Werke (Werke, die in diesem Schritt noch nicht angefahren und somit nicht als bestreikt aufgefunden wurden).

Die Bellman-Gleichung für diesen Erwartungswert lautet:

$$E(x, k, U) = \min_{u \in U} (d(x, u) + (1 - p_u) \cdot E(u, k + 1, W_{k+1}) + p_u \cdot E(u, k, U \setminus \{u\})) \quad (10)$$

Erklärung der Zustandsübergänge: Aus der Menge U wird das optimale Werk u gewählt. Die resultierenden Kosten setzen sich additiv aus der Fahrtzeit $d(x, u)$ und dem Erwartungswert der Zukunft zusammen. Mit der Gegenwahrscheinlichkeit $(1 - p_u)$ ist das Werk geöffnet, der Schritt k wird erfolgreich abgeschlossen, und der Prozess geht in die Schicht $k + 1$ mit allen Werken W_{k+1} über. Mit der Wahrscheinlichkeit p_u streikt das Werk. In diesem Fall verbleibt der Prozess in Schicht k , jedoch schrumpft die Menge der wählbaren Alternativen auf $U \setminus \{u\}$.

Basisfälle: Nach dem letzten erfolgreichen Arbeitsschritt ist lediglich die verbleibende Fahrt zum Ziel T notwendig:

$$E(u, m + 1, W_{m+1}) = d(u, T) \quad (11)$$

Sind für einen Arbeitsschritt keine fähigen Werke mehr übrig, explodieren die Kosten ins Unendliche, da die Mission gescheitert ist:

$$E(x, k, \emptyset) = \infty \quad (12)$$

4.2.2.1 Herleitung der Zustandsübergänge (Bellman-Gleichung)

Die präsentierte rekursive Gleichung basiert auf dem Gesetz der totalen Erwartung (Law of Total Expectation) aus der Wahrscheinlichkeitstheorie in Kombination mit dem Optimalitätsprinzip von Bellman.

Für einen gegebenen Zustand (x, k, U) suchen wir diejenige Entscheidung für ein Werk $u \in U$, die den Erwartungswert der verbleibenden Gesamtdauer minimiert. Sobald die Entscheidung für ein konkretes Werk u gefallen ist, unterliegt das System einem stochastischen Zufallsexperiment mit zwei disjunkten Ereignissen:

- **Ereignis O_u (Offen):** Das Werk streikt nicht. Die Wahrscheinlichkeit hierfür ist $P(O_u) = 1 - p_u$.
- **Ereignis S_u (Streik):** Das Werk streikt. Die Wahrscheinlichkeit hierfür ist $P(S_u) = p_u$.

Nach dem Gesetz der totalen Erwartung setzt sich der bedingte Erwartungswert der Restzeit bei der initialen Wahl von u , den wir $E(x, k, U \mid u)$ nennen, aus der Summe der Erwartungswerte der Einzelergebnisse, gewichtet mit ihrer jeweiligen Eintrittswahrscheinlichkeit, zusammen:

$$E(x, k, U \mid u) = P(O_u) \cdot E(x, k, U \mid u, O_u) + P(S_u) \cdot E(x, k, U \mid u, S_u) \quad (13)$$

Die Kosten, die in beiden Ereignisfällen deterministisch und unvermeidbar anfallen, sind die Fahrzeit $d(x, u)$ vom aktuellen Standort x zum Zielwerk u . Die zukünftigen erwarteten Restzeiten unterscheiden sich jedoch je nach eingetretenem Ereignis:

- Tritt O_u ein, ist der Arbeitsschritt abgeschlossen. Das System geht in den Zustand $(u, k + 1, W_{k+1})$ über.
- Tritt S_u ein, bleibt der Arbeitsschritt unvollendet. Das System verbleibt in der Schicht k , geht aber in den reduzierten Zustand $(u, k, U \setminus \{u\})$ über.

Setzt man diese Zusammenhänge in die obige Formel ein, ergibt sich für eine fest gewählte Entscheidung u :

$$E(x, k, U \mid u) = (1 - p_u) \cdot (d(x, u) + E(u, k + 1, W_{k+1})) + p_u \cdot (d(x, u) + E(u, k, U \setminus \{u\})) \quad (14)$$

Da die Fahrzeit $d(x, u)$ unabhängig vom Eintreten des Streiks anläuft, kann sie ausmultipliziert und zusammengefasst werden (da $(1 - p_u) + p_u = 1$). Der Term vereinfacht sich somit zu:

$$E(x, k, U \mid u) = d(x, u) + (1 - p_u) \cdot E(u, k + 1, W_{k+1}) + p_u \cdot E(u, k, U \setminus \{u\}) \quad (15)$$

Nach dem Optimalitätsprinzip der Dynamischen Programmierung muss eine globale Strategie in jedem Teilschritt die lokal optimale Entscheidung treffen. Um die bestmögliche Gesamtstrategie zu finden, muss folglich der Erwartungswert über alle noch wählbaren Werke $u \in U$ minimiert werden. Dies führt direkt zur finalen Formulierung des Markov-Entscheidungsprozesses:

$$E(x, k, U) = \min_{u \in U} E(x, k, U \mid u) \quad (16)$$

4.2.2.2 Bewertung von Modell A

Vorteile: Dieses Modell punktet durch eine vergleichsweise kompakte Zustandsrepräsentation. Es liefert eine Policy, die im statistischen Alltag sehr effizient ist.

Nachteile: Der Erwartungswert ignoriert Extremfälle. Eine Route könnte einen extrem guten Durchschnitt aufweisen, aber in 5% der Fälle eine extreme Verzögerung verursachen, die die vorgegebene Frist L übersteigt. Für zeitkritische Einzelmissionen, wie den Weltraumauftrag, ist dieser Ansatz daher riskant.

4.2.3 Modell B: Maximierung der Fristeinhaltung (Erfolgswahrscheinlichkeit)

Um der Problemstellung Aufgabe – dem einzuhaltenden Transport eines Geräts für einen Weltraumstart – gerecht zu werden, muss die strikte Zeitfrist L als Restriktion in die Optimierung einfließen. Das Ziel ist nicht die Minimierung der Fahrtzeit an sich, sondern die Maximierung der Wahrscheinlichkeit, dass die Fahrtdauer am Ende kleiner oder gleich L ist.

Hierfür erweitern wir den Zustandsraum um eine Variable t , welche die noch verbleibende Zeit bis zum Verstreichen der Frist L repräsentiert. Wir definieren $P(x, k, U, t)$ als die maximale Wahrscheinlichkeit, das finale Ziel T innerhalb der Restzeit t erfolgreich zu erreichen.

Die Optimierungsgleichung ändert sich somit signifikant:

$$P(x, k, U, t) = \max_{u \in U} ((1 - p_u) \cdot P(u, k + 1, W_{k+1}, t - d(x, u)) + p_u \cdot P(u, k, U \setminus \{u\}, t - d(x, u))) \quad (17)$$

Erklärung: Anstatt Distanzen zu addieren, wird nun das Produkt aus Überlebenswahrscheinlichkeiten maximiert. Die Fahrt zum gewählten Werk u verbraucht exakt $d(x, u)$ Zeiteinheiten, weshalb die Restzeit in beiden Rekursionszweigen auf $t - d(x, u)$ abnimmt.

Basisfälle: Hat der LKW alle Schritte abgeschlossen, ist die Mission genau dann erfolgreich (Wahrscheinlichkeit 1), wenn die Fahrt zum Ziel T in der noch verbleibenden Zeit t machbar ist:

$$P(u, m+1, W_{m+1}, t) = \begin{cases} 1 & \text{falls } t \geq d(u, T) \\ 0 & \text{sonst} \end{cases} \quad (18)$$

Ist die Zeit abgelaufen ($t < 0$) oder sind alle Alternativwerke erschöpft ($U = \emptyset$), sinkt die Erfolgswahrscheinlichkeit auf 0:

$$P(x, k, U, t) = 0 \quad \text{für } t < 0 \quad \text{oder} \quad U = \emptyset \quad (19)$$

4.2.4 Herleitung der Zustandsübergänge (Gesetz der totalen Wahrscheinlichkeit)

Die Herleitung für Modell B verläuft strukturell analog zu Modell A, basiert jedoch auf dem Gesetz der totalen Wahrscheinlichkeit anstelle der totalen Erwartung. Außerdem ändert sich die Zielsetzung: Anstatt eine aufaddierte Kostenfunktion zu minimieren, muss ein Wahrscheinlichkeitswert maximiert werden.

Für einen gegebenen Zustand (x, k, U, t) mit der verbleibenden Restzeit t betrachten wir die isolierte Entscheidung, das LKW-Routing zum Werk $u \in U$ fortzusetzen. Auch hier unterliegt das System nach der Wahl von u einem stochastischen Zufallsexperiment mit den zwei disjunkten Ereignissen:

- **Ereignis O_u (Offen):** Das Werk streikt nicht ($P(O_u) = 1 - p_u$).
- **Ereignis S_u (Streik):** Das Werk streikt ($P(S_u) = p_u$).

Sei $P(x, k, U, t \mid u)$ die bedingte Wahrscheinlichkeit, das Ziel rechtzeitig zu erreichen, unter der festen Voraussetzung, dass wir uns im aktuellen Schritt für Werk u entscheiden. Nach dem Gesetz der totalen Wahrscheinlichkeit ergibt sich dieser Wert aus den mit ihren Eintrittswahrscheinlichkeiten gewichteten Erfolgsaussichten der beiden Folgezustände:

$$P(x, k, U, t \mid u) = P(O_u) \cdot P(x, k, U, t \mid u, O_u) + P(S_u) \cdot P(x, k, U, t \mid u, S_u) \quad (20)$$

Der fundamentale Unterschied zu Modell A liegt in der Verarbeitung der Fahrzeit $d(x, u)$. Da es sich bei der zu optimierenden Metrik um eine Wahrscheinlichkeit (einen Wert zwischen 0 und 1) und nicht um eine Fahrdauer handelt, kann $d(x, u)$ nicht einfach additiv vor die Gleichung gezogen werden. Stattdessen fungiert die Zeit als limitierende Ressource, die bei jedem physischen Ortswechsel zwingend verbraucht wird. Die verbleibende Restzeit für alle Folgezustände schrumpft auf $t - d(x, u)$.

Eingesetzt in das Totalitätsgesetz ergibt sich für die beiden Ereignisse:

- Tritt O_u ein, wird der Schritt erfolgreich beendet und das System überführt in den Zustand $(u, k+1, W_{k+1}, t - d(x, u))$.
- Tritt S_u ein, bleibt der LKW in Schicht k am geschlossenen Werk u stehen und muss aus der reduzierten Menge $(u, k, U \setminus \{u\}, t - d(x, u))$ ein neues Ausweichwerk suchen.

Daraus resultiert die Gleichung für die isolierte Entscheidung u :

$$P(x, k, U, t \mid u) = (1 - p_u) \cdot P(u, k+1, W_{k+1}, t - d(x, u)) + p_u \cdot P(u, k, U \setminus \{u\}, t - d(x, u)) \quad (21)$$

Gemäß dem Optimalitätsprinzip von Bellman erfordert eine global optimale Strategie die Maximierung der Erfolgswahrscheinlichkeit in jedem Teilschritt. Um die beste Gesamtevaluation für den Zustand (x, k, U, t) zu finden, iteriert der Markov-Entscheidungsprozess über alle wählbaren Werke in U und wählt das Maximum:

$$P(x, k, U, t) = \max_{u \in U} P(x, k, U, t \mid u) \quad (22)$$

4.2.5 Begründung für die Wahl von Modell B

Für das vorliegende Weltraum-Experiment wird Modell B als Erweiterung gewählt. Ein Erwartungswert (Modell A) deutet an, dass man Zeitgewinne bei einem frühen Start gegen Zeitverluste bei einem späten Start aufrechnen könnte. Trifft jedoch das Gerät auch nur eine Stunde nach Verstreichen der Frist L ein, so kann die Rakete nicht starten und die Mission scheitert. Ob man 10 Tage oder 10 Minuten vor der Frist L ankommt, macht für den Erfolg der Mission keinen Unterschied, solange L eingehalten wird.

Modell B ist ein risiko-bewusster Algorithmus. Er ist in der Lage, eine Route zu wählen, deren Fahrtweg im Normalfall vielleicht deutlich länger ist als bei anderen Routen, die aber über Ausweichwerke verfügt, welche eine Garantieschranke für L bieten. Die Integration der verbleibenden Zeit t in den Zustandsraum

erhöht zwar die asymptotische Komplexität drastisch, ist aber eine valide und elegante Methode, um die probabilistische Stabilität von Deadlines exakt zu garantieren.

4.2.6 Generierung der Streikwahrscheinlichkeiten

Um die probabilistische Erweiterung auf die originalen, deterministischen Beispiele anwenden zu können, ohne die Struktur der Eingabedateien zu verändern, werden die Streikwahrscheinlichkeiten p_u zur Laufzeit generiert.

Hierbei kommt der C++ Zufallszahlengenerator `std::mt19937` zum Einsatz. Jedem registrierten Werk u wird initial eine gleichverteilte Streikwahrscheinlichkeit $p_u \in [0.05, 0.30]$ zugewiesen. Um die Reproduzierbarkeit der Testfälle zu gewährleisten, wird der Generator mit einem konstanten Seed initialisiert. Das Programm agiert somit pseudo-zufällig: Die Wahrscheinlichkeiten variieren stark zwischen den einzelnen Werken, bleiben aber bei jedem Programmneustart für dasselbe Werk identisch.

4.2.7 Umsetzung

Die Einführung der stochastischen Variablen und insbesondere der kontinuierlichen Restzeit t erfordert einen Paradigmenwechsel in der Implementierung. Der deterministische Basis-Algorithmus nutzte zweidimensionale `std::vector`-Konstrukte. Ein Transfer dieses iterativen Bottom-Up-Ansatzes auf das stochastische Modell B würde eine vierdimensionale Matrix der Größe $N \times m \times 2^{W_{\max}} \times L$ erzwingen. Dies führt zu einer Überschreitung des verfügbaren Arbeitsspeichers.

Daher wird der Lösungsansatz auf eine Top-Down Dynamische Programmierung mit Memoization (Caching) umstrukturiert. Dies macht sich die Eigenschaft zunutze, dass der reale Zustandsraum extrem dünn besetzt ist: Die LKW-Fahrerin kann zu einem bestimmten Arbeitsschritt an einem bestimmten Werk physikalisch bedingt nur eine stark begrenzte Menge an Restzeiten t aufweisen.

4.2.7.1 Zustandsraum und Datenstrukturen

Jeder zu evaluierende Knotenpunkt im Entscheidungsbaum des Markov-Entscheidungsprozesses wird durch vier Parameter eindeutig definiert. Zur sauberen Kapselung wird das C++-Konstrukt `std::tuple` verwendet:

```
using State = std::tuple<int, int, int, long long>;
```

Die Elemente des Tupels repräsentieren in dieser Reihenfolge:

1. x : Die ID des aktuellen Aufenthaltsortes.
2. k : Den Index des aktuell auszuführenden Arbeitsschrittes.
3. mask : Eine Bitmaske der noch nicht bestreikten Werke für Schritt k .
4. t : Die verbleibende Zeit bis zum Verstreichen der Frist L .

4.2.7.1.1 Effiziente Mengenverwaltung via Bitmasking

Die Restmenge an fähigen Werken $U \subseteq W_k$ muss bei jedem rekursiven Aufruf übergeben und potenziell modifiziert werden. Eine Repräsentation als `std::vector` oder `std::set` würde einen enormen Overhead durch Kopieroperationen und dynamische Speicherallokationen erzeugen.

Stattdessen wird die Menge als 32-Bit Integer (`int mask`) abgebildet. Jedes Bit repräsentiert den Zustand eines Werkes (1 = verfügbar, 0 = bestreikt oder bereits ausgeschlossen). Tritt der Fall ein, dass ein Werk i bestreikt wird, lässt sich die neue Restmenge $U \setminus \{u\}$ in einer einzigen CPU-Taktzyklus-Operation mithilfe des XOR-Operators berechnen: `int fallback_mask = mask ^ (1 << i);`

4.2.7.2 Caching der Erfolgswahrscheinlichkeiten

Um die überlappenden Teilprobleme der Dynamischen Programmierung nicht mehrfach zu berechnen, wird eine Caching-Struktur benötigt.

Hierfür kommt die Datenstruktur `std::map<State, double> memo` zum Einsatz. Sie ordnet jedem berechneten Zustand die maximale Erfolgswahrscheinlichkeit zu. Parallel dazu existiert eine `std::map<State, int> best_action`, welche für jeden Zustand speichert, welches Werk u als Nächstes angefahren werden muss, um diese Wahrscheinlichkeit zu realisieren. Wird die rekursive Funktion `get_prob` aufgerufen, prüft sie zunächst über `memo.count(state)`, ob dieser exakte Pfadverlauf in der Vergangenheit bereits evaluiert

wurde. Ist dies der Fall, wird die Wahrscheinlichkeit in $\mathcal{O}(\log(\text{Zustandsanzahl}))$ aus der Map geladen, was Rekursionstiefen stark minimiert.

4.2.7.3 Extraktion und Konstruktion der Policy

Im Gegensatz zum deterministischen Modell, das einen einzelnen eindimensionalen Pfad (Master-Plan) ausgibt, generiert die stochastische Erweiterung einen mehrdimensionalen Entscheidungsbaum (eine Policy).

Die Rekonstruktion nach Abschluss der DP-Phase teilt sich in zwei Darstellungsformen:

1. **Die Hauptroute:** Ausgehend vom Start S wird der Baum durchlaufen unter der Annahme, dass kein einziges Werk streikt. In jedem Schritt wird die volle Bitmaske an die Folgezustände übergeben und das zugehörige Werk aus `best_action` rausgenommen.
2. **Die Fallbacks:** Für jeden Knoten auf der Hauptroute wird simuliert, was passieren würde, wenn genau dieses Werk streikt. Durch die Abfrage des Zustands mit der invertierten Bitmaske am entsprechenden Knoten generiert das Programm genaue Ausweichinstruktionen (z. B. „Wenn Werk A streikt, fahre zu Werk C“).

Diese Architektur erfüllt die Anforderung der Aufgabenstellung, sowohl eine Standardroute als auch alternative Streckenplanungen auszugeben, erweitert um das Wissen über die maximale stochastische Erfolgswahrscheinlichkeit.

4.2.8 Laufzeit- und Speicherplatzanalyse der stochastischen Erweiterung

Die Einführung der stochastischen Erfolgswahrscheinlichkeit und der verbleibenden Zeit t in den Zustandsraum transformiert das Problem. Während der deterministische Ansatz in polynomieller Zeit lösbar ist, weist das erweiterte Modell (Modell B) theoretische Laufzeitgrenzen auf, die exponentielle und pseudo-polynomielle Faktoren beinhalten.

Für die Analyse gelten folgende Definitionen:

- N : Anzahl aller Werke im Graphen.
- E : Anzahl der Straßen.
- m : Anzahl der sequentiellen Arbeitsschritte.
- $W_{\max} = \max(|W_k|)$: Die maximale Anzahl fähiger Werke für einen einzelnen Arbeitsschritt.
- L : Die vorgegebene zeitliche Frist.
- S : Die Menge der tatsächlich in der Caching-Map evaluierten Zustände.

4.2.8.1 Laufzeitanalyse

Die Gesamtlaufzeit setzt sich weiterhin additiv aus der Vorberechnung (Dijkstra) und der dynamischen Programmierung zusammen.

Da die Fahrzeiten zwischen den Knoten von den Streikwahrscheinlichkeiten unabhängig sind, bleibt die Initialphase exakt gleich. Die iterative Anwendung des Dijkstra-Algorithmus mit einem Min-Heap benötigt deterministisch $\mathcal{O}(N \cdot E \log N)$.

Worst-Case: Der rekursive Algorithmus erkundet den theoretischen Zustandsraum. Ein Zustand (x, k, mask, t) setzt sich zusammen aus:

- x : Dem aktuellen Standort (maximal $W_{\max}$ Möglichkeiten, da $x \in W_{k-1}$).
- k : Dem Arbeitsschritt (maximal m Möglichkeiten).
- mask : Der Bitmaske verfügbarer Werke für Schritt k (maximal $2^{W_{\max}}$ Möglichkeiten).
- t : Der verbleibenden Zeit (maximal L Möglichkeiten).

In jedem Zustand iteriert das Programm über bis zu $W_{\max}$ verfügbare Werke, um den stochastischen Erwartungswert des nächsten Übergangs zu berechnen. Da die Zustände in einer `std::map` (Rot-Schwarz-Baum) zwischengespeichert werden, kostet jeder Lese-/Schreibzugriff $\mathcal{O}(\log|S|)$. Die theoretische Worst-Case-Laufzeit der DP-Phase beträgt somit:

$$\mathcal{O}(m \cdot W_{\max}^2 \cdot 2^{W_{\max}} \cdot L \cdot \log|S|) \quad (23)$$

Diese Komplexitätsklasse offenbart zwei kritische Wachstumsfaktoren: Sie ist exponentiell bezüglich der Anzahl der Ausweichwerke ($2^{W_{\max}}$) und pseudo-polynomiell bezüglich der Zeitfrist L .

Average-Case: Die gute Performance des Algorithmus in der Praxis ergibt sich daraus, dass der Worst-Case nie eintritt. Der Zustandsraum ist extrem ausgedünnt:

1. Zeitstempel: Die Restzeit t nimmt niemals alle Integer-Werte von 0 bis L an. Sie existiert nur in den exakten Subtraktionen der realen Fahrtauern $d(x, u)$.
2. Massives Time-Pruning: Pfade, bei denen die Fahrt zu einem Werk die Restzeit negativ werden lässt ($t < d(x, u)$), werden in $\mathcal{O}(1)$ abgebrochen, bevor Folgezustände in den Cache geschrieben werden.

Im durchschnittlichen Fall skaliert die Laufzeit daher nur mit einem winzigen Bruchteil des theoretischen Limits, wodurch selbst komplexe Testfälle in wenigen Sekunden terminiert werden.

Best-Case: Der beste Fall tritt ein, wenn die Frist L so restriktiv ist, dass bereits die Fahrt von S zu den Werken des ersten Arbeitsschritts unmöglich ist ($d(S, u) > L$). Der Algorithmus verwirft alle Zweige sofort an der Wurzel. Die Laufzeit beschränkt sich in diesem Fall fast exakt auf die APSP-Vorberechnung: $\mathcal{O}(N \cdot E \log N + W_{\max})$.

4.2.8.2 Speicherplatzanalyse

Der RAM-Verbrauch wird durch zwei primäre Datenstrukturen dominiert: Die Distanzmatrix und die Caching-Map der Memoization.

Die Distanzmatrix: Wie im deterministischen Basis-Algorithmus wird eine $N \times N$ Lookup-Tabelle als zusammenhängender `std::vector` alloziert. Diese benötigt deterministisch quadratischen Speicherplatz in $\mathcal{O}(N^2)$, was bei typischen Logistiknetzwerken unproblematisch ist.

Der Memoization-Cache: Die Map `memo` speichert jeden eindeutigen Zustand als Schlüssel und einen `double`-Wert als Erfolgswahrscheinlichkeit. Im Gegensatz zu einer Matrix alloziert die Map nur Speicher für Zustände, die der Algorithmus tatsächlich besucht. Im absoluten Worst-Case würde der Speicherbedarf asymptotisch dem Zustandsraum entsprechen: $\mathcal{O}(m \cdot W_{\max} \cdot 2^{W_{\max}} \cdot L)$.

Durch die Top-Down-Architektur bleibt der Speicher im **Average-Case** jedoch streng proportional zur Menge der physikalisch sinnvollen Pfade, die L nicht reißen. Da ungültige Pfade gar nicht erst in untere Schichten expandieren, bewahrt die Kombination aus Bitmasking und On-the-Fly Pruning das System davor, die kritische Grenze des Arbeitsspeichers zu erreichen.

4.2.8.3 Interpretation einer Ausgabe (Beispiel: `lieferung03.txt`)

Ein interessantes Resultat der stochastischen Erweiterung (Modell B) zeigt sich bei der Auswertung des Beispiels `lieferung03.txt`. Im originalen, deterministischen Programm wurde dieser Testfall korrekterweise als „UNMOEGlich“ deklariert. Das bedeutet, es gab keine einzige Route im Straßennetz, die im theoretischen Worst-Case (bei einem maximal ungünstigen Streik) die Frist $L = 285$ garantieren konnte.

Anmerkung: Es wurden alle Beispiele von 00 bis 06, sowie 09 ausgeführt und in den Abgabeordner eingefügt („lieferungxx_stochastisch_out.txt“). Die restlichen Dateien konnten aufgrund der Laufzeit- und Speicherkomplexität nicht gelöst werden. Das genutzte Programm heißt `wahrscheinlichkeit.cpp`.

Der stochastische Algorithmus hingegen liefert folgende Ausgabe:

Output (Auszug):

MOEGLICH

Maximale Erfolgswahrscheinlichkeit (Frist L=285): 45.06%

— ENTSCHEIDUNGS-POLICY —

Hauptroute (Keine Streiks): S -> [4 (24.9% Streikrisiko)] -> [19 (16.5% Streikrisiko)] -> [21 (8.6% Streikrisiko)] -> [32 (5.2% Streikrisiko)] -> [20 (5.6% Streikrisiko)] -> [26 (6.2% Streikrisiko)] -> [12 (29.3% Streikrisiko)] -> [28 (14.6% Streikrisiko)] -> [18 (5.3% Streikrisiko)] -> [13 (5.4% Streikrisiko)] -> T

Fallbacks (Wenn ein Werk der Hauptroute streikt):
 Schritt 1: Falls [4] streikt -> Weiche aus auf [7]
 Schritt 2: Falls [19] streikt -> Weiche aus auf [1]
 Schritt 3: Falls [21] streikt -> Weiche aus auf [16]
 ...

- **Vom Worst-Case zur Wahrscheinlichkeit (45.06%):** Der Algorithmus stimmt der deterministischen Lösung insoweit zu, als dass eine absolute Garantie (100 %) unmöglich ist. Anstatt das Problem jedoch als unlösbar abzuweisen, sucht das stochastische Modell den bestmöglichen Kompromiss. Der Wert von 45.06 % ist das globale Maximum der Funktion $P(S, 1, \text{mask}, 285)$. Das bedeutet: Wenn die LKW-Fahrerin exakt den Anweisungen dieser generierten Policy folgt, wird sie in knapp 45 von 100 Instanzen das Zielgebiet T erreichen, bevor die Frist $L = 285$ abläuft.
- **Die Hauptroute:** Die Hauptroute zeigt den Pfad an, der gefahren wird, wenn alle angefahrenen Werke geöffnet sind. Bemerkenswert ist, dass der Algorithmus hierbei kontinuierlich zwischen Fahrzeit und lokalem Streikrisiko abwägt. So wird in Schritt 1 das Werk 4 gewählt, obwohl es ein hohes Ausfallrisiko von 24.9% besitzt. Der Algorithmus hat mathematisch evaluiert, dass diese Wahl – gekoppelt an die exzellente Position des Ersatzwerks 7 im Falle eines Streiks – global die höchste kumulierte Überlebenswahrscheinlichkeit bietet.
- **Dynamische Fallbacks:** Im Gegensatz zum linearen Master-Plan der Originalaufgabe generiert die MDP-Lösung einen reaktiven Entscheidungsbaum. Die Anweisung „Falls [4] streikt -> Weiche aus auf [7]“ ist kein statischer Ratschlag, sondern in die 45.06 % einkalkuliert. Der Algorithmus weiß bereits zum Startzeitpunkt, dass wenn beispielsweise Werk 4 verschlossen ist, die Fahrt zu Werk 7 wertvolle Zeit kostet, weshalb die Restzeit t drastisch sinkt. Dennoch reicht diese reduzierte Restzeit aus, um das Gerät mit der berechneten Restwahrscheinlichkeit noch rechtzeitig abzuliefern.

4.2.9 Theoretische Robustheit gegenüber mehreren Ausfällen

Ein wesentlicher theoretischer Aspekt der vorliegenden Implementierung ist, dass das zugrundeliegende mathematische Modell der stochastischen dynamischen Programmierung bereits eine Absicherung gegen mehrfache, sequentielle Ausfälle innerhalb desselben Arbeitsschritts k beinhaltet. Obwohl die Programmausgabe im Sinne der Aufgabenstellung lediglich eine primäre Ausweichroute w'_k für jedes im Normalplan enthaltene Werk w_k explizit benennt, „weiß“ der Algorithmus durch die rekursive Struktur der Bellman-Gleichung bereits, wie im Falle eines Streiks des Ausweichwerks zu verfahren ist.

Der Zustand in der DP-Tabelle ist formal über das Tripel (k, u, U) definiert, wobei $U \subseteq W_k$ die Menge der zum aktuellen Zeitpunkt noch nicht bestreikten und fähigen Werke darstellt. Wenn das primäre Ausweichwerk w'_k ebenfalls ausfällt, reduziert sich die Entscheidungssituation mathematisch auf den Zustand $(k, w'_k, U \setminus \{w'_k\})$. Da die Kosten $E(k, u, U)$ für alle möglichen Teilmengen U vorberechnet werden, ist die optimale Reaktion auf einen Zweit- oder Drittstreik bereits in den Datenstrukturen hinterlegt. Die aktuelle Ausgabe beschränkt sich aus Gründen der Übersichtlichkeit auf die Darstellung der ersten Alternative („Plan B“). Technisch gesehen generiert der Algorithmus jedoch keinen statischen Pfad, sondern eine vollständige Entscheidungsstrategie (Policy): Sollte eine Kette von Werken nacheinander ausfallen, könnte das Programm sukzessive die jeweils nächstbeste Option aus der Menge U wählen, bis entweder ein funktionsfähiges Werk gefunden oder die Menge U vollständig erschöpft ist ($U = \emptyset$). In letzterem Fall erkennt das System die Unlösbarkeit der Mission unter den gegebenen Zeitrestriktionen ($E = \infty$).

4.2.10 Fazit zur Erweiterung

Die Transformation zu einem stochastischen Markov-Entscheidungsprozess zwingt das Programm in eine exponentielle und pseudo-polynomielle Komplexitätsklasse. Dennoch beweist die Implementierung mittels Top-Down-Memoization, Bit-Arithmetik und aggressivem Pruning, dass selbst mathematisch komplexe Risikoabschätzungen in Logistiknetzwerken berechnet werden können, solange die maximale Anzahl der Ausweichalternativen ($W_{\max}$) pro Arbeitsschritt innerhalb realistischer Grenzen bleibt.

5 Werkzeuge

5.1 Entwicklungsumgebung

Die Algorithmen wurden vollständig in der Programmiersprache C++ entwickelt, um durch manuelle Speicherverwaltung und systemnahe Ausführung eine maximale Performance zu gewährleisten. Als Texteditor kam Zed zum Einsatz. Die Kompilierung und Ausführung des Codes erfolgten nativ auf einem Linux-Betriebssystem unter Verwendung des g++-Compilers mit folgendem Command: `g++ -std=c++17 -O3 -march=native -flto -funroll-loops -fomit-frame-pointer -DNDEBUG -pthread DATEINAME.cpp -o DATEINAME`.

5.2 Einsatz künstlicher Intelligenz (Gemini Pro 3.1)

Um den Entwicklungsprozess effizienter zu gestalten und komplexe konzeptionelle Hürden zu überwinden, wurde das Large Language Model Gemini Pro 3.1 als Hilfe in die Arbeit eingebunden. Die Nutzung gliederte sich dabei in vier Phasen:

1. **Recherche:** Zu Beginn der Projektphase diente die KI als interaktive Suchmaschine. Sie wurde genutzt, um einen schnellen Überblick über etablierte graphentheoretische Algorithmen zu gewinnen und die zugrunde liegende Problemstellung mit bekannten Problemen abzugleichen.
2. **Code-Review und Strukturierung:** Während der aktiven Programmierphase wurde das Modell für Code-Review eingesetzt.
3. **Dokumentation und theoretische Verifikation:** In der abschließenden Phase wurde das Modell ergänzend genutzt, um den Sprachstil und die Formatierung in Typst zu verfeinern. Außerdem diente die KI als Kontrollinstanz für mathematische Ausdrücke.

6 Beispiele

Aufgrund des großen Umfangs der Ausgaben bei den meisten Pflichtbeispielen sind die vollständigen Ergebnisdateien nur im Abgabeordner hinterlegt (außer bei 00, 01, 02 und 03). Die restlichen Ausgabe-dateien sind im Abgabeordner aufzufinden

Die Pflichtbeispiele wurden mit dem Programm `main.cpp` auf einem System mit den folgenden Hardware-Spezifikationen ausgeführt:

- **Prozessor (CPU):** 12th Gen Intel Core i7-1255U (10 Kerne, bis zu 4.70 GHz)
- **Grafikkarte (GPU):** Intel Iris Xe Graphics (Integrated)
- **Betriebssystem:** Omarchy Linux

6.1 Pflichtbeispiele

6.1.1 lieferung00.txt

Output:

```
MOEGlich
15
9
S [3 1] [5 2] T
11
3 [2 1] 3 [5 2] T
15
5 T [4 2] T
```

6.1.2 lieferung01.txt

Output:

```
MOEGlich
40
26
S [1 1] [4 2] [7 3] T
40
1 S [2 1] S 1 [4 2] [7 3] T
37
4 [5 2] [8 3] T
38
7 T [6 3] T
```

6.1.3 lieferung02.txt

Output:

```
MOEGlich
188
122
S [1 1] S [2 2] [4 3] 2 13 7 [10 4] 7 [13 5] 2 S 1 T
138
1 6 [5 1] [3 2] [4 3] 2 13 7 [10 4] 7 [13 5] 2 S 1 T
157
```

```
2 13 7 5 [3 2] [4 3] 2 13 7 [10 4] 7 [13 5] 2 S 1 T
188
4 3 5 6 12 [11 3] 9 8 [10 4] 7 [13 5] 2 S 1 T
174
10 7 5 [6 4] 5 7 [13 5] 2 S 1 T
124
13 [7 5] 13 2 S 1 T
```

6.1.4 lieferung03.txt**Output:**

UNMOEGlich

6.1.5 lieferung04.txt**Output:**

MOEGlich

...

6.1.6 lieferung05.txt**Output:**

MOEGlich

...

6.1.7 lieferung06.txt**Output:**

UNMOEGlich

6.1.8 lieferung07.txt**Output:**

MOEGlich

...

6.1.9 lieferung08.txt**Output:**

MOEGlich

...

6.1.10 lieferung09.txt**Output:**

MOEGLICH

...

6.1.11 lieferung10.txt**Output:**

MOEGLICH

...

6.2 Eigene Beispiele**6.2.1 greedy_falle.txt**

Dieses eigens konstruierte Beispiel verdeutlicht die Notwendigkeit der vorausschauenden Dynamischen Programmierung. In der Topologie dieses Graphen liegt das Werk 1 extrem nah am Startpunkt S (Kosten: 2). Ein naiver Greedy-Ansatz würde dieses Werk als erstes Ziel für den Arbeitsschritt 1 auswählen. Werk 1 liegt jedoch stark isoliert auf einer Art Halbinsel: Wird es bestreikt, dauert die Fahrt zu den Ersatzwerken 2 oder 3 unverhältnismäßig lange (Kosten: 100), wodurch die vorgegebene Frist von $L = 30$ zwangsläufig gerissen wird (siehe Abbildung 2).

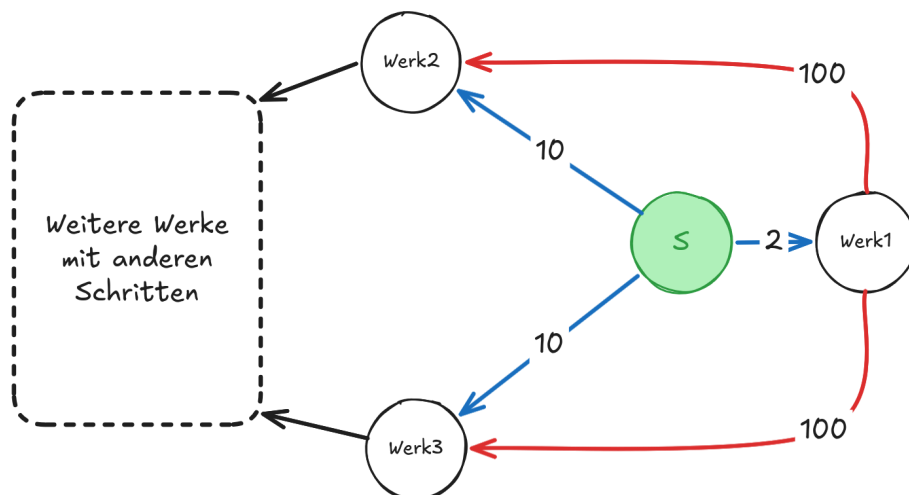


Abbildung 2: Topologie der Greedy-Falle: Die gierige Wahl von Werk 1 führt bei einem Streik zwingend zum Scheitern der Mission

Das Beispiel beweist, dass der auf Dynamischer Programmierung basierende Algorithmus – im Gegensatz zu einem Greedy-Ansatz – Täuschungen durch vorausschauendes Pruning lokaler Minima umgeht und den global krisensichersten Master-Plan ermittelt.

Output:

MOEGLICH

14

6

S [1 1] [4 2] T

14

```
1 4 [2 1] [4 2] T
10
4 T [5 2] T
```

6.2.2 speicher_falle.txt

Dieses eigens konstruierte Beispiel („speicher_falle.txt“) verdeutlicht die asymptotischen Grenzen der gewählten Graphen-Repräsentation und der APSP-Vorberechnung. In der Topologie dieses Graphen sind die Knoten extrem dicht miteinander vernetzt, wodurch nahezu eine vollständige Clique entsteht. Ein Algorithmus profitiert in der Praxis normalerweise von stark ausgedünnten Straßennetzen. Dieser Graph zwingt das Programm jedoch, in der initialen Dijkstra-Phase quadratisch ansteigende Kantenmengen ($\mathcal{O}(N^2)$) auszuwerten. Da zudem die Zeitfrist mit $L = 100000$ bewusst extrem hoch angesetzt ist, greift das algorithmische Pruning in der Vorwärts-Synthese nicht: Der Suchbaum wird kaum beschnitten, wodurch die verschachtelten Matrizen-Iterationen der DP vollständig evaluiert werden müssen.

Das Beispiel beweist, dass die überragende Performance des Algorithmus maßgeblich auf der Planarität realer Logistiknetzwerke beruht – und zeigt transparent auf, dass der gewählte All-Pairs-Shortest-Path-Ansatz bei einem rein theoretischen, vollständig vernetzten Worst-Case-Szenario unvermeidlich an seine absoluten Speicher- und Laufzeitgrenzen stößt.

6.2.3 alternativen_explosion.txt (MDP-Erweiterung)

Dieses Beispiel beweist den in der Laufzeitanalyse hergeleiteten Bottleneck der stochastischen Erweiterung: Die exponentielle Abhängigkeit von der maximalen Anzahl der Ausweichwerke.

Der Graph besteht aus nur drei Arbeitsschritten, wobei Arbeitsschritt 2 exakt 30 fähige Werke auf engstem Raum bietet. Der deterministische Basis-Algorithmus löst diese Instanz in wenigen Millisekunden, da er lediglich eine quadratische Matrix (30×30) iteriert. Übergibt man dieses Beispiel jedoch der stochastischen Erweiterung, versucht die Top-Down-DP, alle möglichen Streik-Kombinationen für Schritt 2 in der Bitmaske abzubilden. Der Memoization-Cache sprengt deswegen die RAM-Kapazitäten des Systems, bevor die Rekursionstiefe erreicht ist.

Dieses Beispiel zeigt das Skalierungsproblem in der stochastischen Methode und beweist, dass das Bitmask-Pruning nur dann nützlich ist, solange $W_{\max}$ eine kritische Grenze von wenigen (zweistelligen) Alternativen nicht überschreitet.

7 Quellcode

Der nachfolgende C++-Code repräsentiert die beschriebene Implementierung in der Datei *main.cpp*.

```
#include <iostream>
#include <vector>
#include <string>
#include <queue>
#include <chrono>

using namespace std;

const long long INF = 1e15;

void solve() {
    long long L; // Zeitfrist
    cin >> L;
    int m; // Anzahl Arbeitsschritte
    cin >> m;

    // Anzahl der Werke pro Schritt
    vector<int> counts(m);
    for(int i = 0; i < m; ++i) {
        cin >> counts[i];
    }

    int w; // Anzahl aller Werke
    cin >> w;

    // Werke zwischenspeichern
    vector<pair<string, int>> temp_works(w);
    int max_node_id = 0;
    for(int i = 0; i < w; ++i) {
        cin >> temp_works[i].first >> temp_works[i].second;
        if(temp_works[i].first != "S" && temp_works[i].first != "T") {
            max_node_id = max(max_node_id, stoi(temp_works[i].first));
        }
    }

    int edges_cnt; // Anzahl der Straßen
    cin >> edges_cnt;

    // Straßen zwischenspeichern und maximale ID updaten
    struct EdgeInput {
        string u, v;
        long long t;
    };
    vector<EdgeInput> temp_edges(edges_cnt);
    for(int i = 0; i < edges_cnt; ++i) {
        cin >> temp_edges[i].u >> temp_edges[i].v >> temp_edges[i].t;
        if(temp_edges[i].u != "S" && temp_edges[i].u != "T") {
            max_node_id = max(max_node_id, stoi(temp_edges[i].u));
        }
        if(temp_edges[i].v != "S" && temp_edges[i].v != "T") {
            max_node_id = max(max_node_id, stoi(temp_edges[i].v));
        }
    }

    int n = max_node_id + 2;
    vector<string> node_names(n, "");
    node_names[0] = "S";
    node_names[n - 1] = "T";
    for(int i = 1; i <= max_node_id; ++i) {
```

```

    node_names[i] = to_string(i);
}

auto get_id = [&](const string &s) -> int {
    if (s == "S") return 0;
    if (s == "T") return n - 1;
    return stoi(s);
};

int S_id = get_id("S"); // Start
int T_id = get_id("T"); // Ziel

// Welches Werk gehört zu welchem Schritt?
vector<vector<int>> works(m + 1);
for(int i = 0; i < w; ++i) {
    int id = get_id(temp_works[i].first);
    works[temp_works[i].second].push_back(id);
}

int N = node_names.size();
struct Edge {
    int to;
    long long weight;
};
vector<vector<Edge>> adj(N);

// Ungerichteten Graphen aus den zwischengespeicherten Kanten aufbauen
for(int i = 0; i < edges_cnt; ++i) {
    int u = get_id(temp_edges[i].u);
    int v = get_id(temp_edges[i].v);
    long long weight = temp_edges[i].t;
    adj[u].push_back({v, weight});
    adj[v].push_back({u, weight});
}

// 1. Dijkstra für alle Paare (Distanzen berechnen)
vector<vector<long long>> dist_to(N, vector<long long>(N, INF));
for(int dst = 0; dst < N; ++dst) {
    dist_to[dst][dst] = 0;
    priority_queue<pair<long long, int>, vector<pair<long long, int>>, greater<>> pq;
    pq.push({0, dst});

    while(!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();

        if (d > dist_to[dst][u]) continue;

        for(auto &edge : adj[u]) {
            int v = edge.to;
            long long weight = edge.weight;
            if(dist_to[dst][u] + weight < dist_to[dst][v]) {
                dist_to[dst][v] = dist_to[dst][u] + weight;
                pq.push({dist_to[dst][v], v});
            }
        }
    }
}

auto dist_uv = [&](int u, int v) {
    return dist_to[v][u];
};

```

```

vector<vector<int>>> next_node(N, vector<int>(N, -1));
for(int u = 0; u < N; ++u) {
    for(int dst = 0; dst < N; ++dst) {
        if(u == dst || dist_to[dst][u] == INF) continue;
        for(auto &edge : adj[u]) {
            int v = edge.to;
            long long weight = edge.weight;
            if(dist_to[dst][v] != INF && weight + dist_to[dst][v] == dist_to[dst][u]) {
                // Numerisches Tie-Breaking
                if(next_node[u][dst] == -1 || v < next_node[u][dst]) {
                    next_node[u][dst] = v;
                }
            }
        }
    }
}

// 2. DP rückwärts
vector<vector<long long>>> MinDist(N, vector<long long>(m + 1, INF));
vector<vector<int>>> NextNormal(N, vector<int>(m + 1, -1));

for(int u : works[m]) {
    MinDist[u][m] = dist_uv(u, T_id);
}

for(int k = m - 1; k >= 1; --k) {
    for(int u : works[k]) {
        for(int v : works[k + 1]) {
            if(dist_uv(u, v) == INF || MinDist[v][k+1] == INF) continue;
            long long cand = dist_uv(u, v) + MinDist[v][k+1];

            if(cand < MinDist[u][k]) {
                MinDist[u][k] = cand;
                NextNormal[u][k] = v;
            } else if(cand == MinDist[u][k]) {
                // Numerisches Tie-Breaking
                if(NextNormal[u][k] == -1 || v < NextNormal[u][k]) {
                    NextNormal[u][k] = v;
                }
            }
        }
    }
}

// 3. DP Streikkosten
vector<vector<long long>>> BestAltDist(N, vector<long long>(m + 1, INF));
vector<vector<int>>> NextAlt(N, vector<int>(m + 1, -1));

for(int k = 1; k <= m; ++k) {
    for(int u : works[k]) {
        for(int v : works[k]) {
            if(u == v) continue;
            if(dist_uv(u, v) == INF || MinDist[v][k] == INF) continue;

            long long cand = dist_uv(u, v) + MinDist[v][k];
            if(cand < BestAltDist[u][k]) {
                BestAltDist[u][k] = cand;
                NextAlt[u][k] = v;
            } else if(cand == BestAltDist[u][k]) {
                // Numerisches Tie-Breaking
                if(NextAlt[u][k] == -1 || v < NextAlt[u][k]) {

```

```

        NextAlt[u][k] = v;
    }
}
}
}

// 4. DP vorwärts
vector<vector<long long>> MinArrival(N, vector<long long>(m + 1, INF));
vector<vector<int>> PrevMain(N, vector<int>(m+1, -1));

for(int u : works[1]) {
    if(dist_uv(S_id, u) == INF) continue;
    long long t = dist_uv(S_id, u);

    if(t + BestAltDist[u][1] <= L && t + MinDist[u][1] <= L) {
        MinArrival[u][1] = t;
    }
}

for(int k = 2; k <= m; ++k) {
    for(int u : works[k]) {
        for(int v: works[k-1]) {
            if(MinArrival[v][k-1] == INF || dist_uv(v, u) == INF) continue;

            long long t = MinArrival[v][k-1] + dist_uv(v, u);

            if(t + BestAltDist[u][k] <= L && t + MinDist[u][k] <= L) {
                if(t < MinArrival[u][k]) {
                    MinArrival[u][k] = t;
                    PrevMain[u][k] = v;
                } else if(t == MinArrival[u][k]) {
                    // Numerisches Tie-Breaking
                    if(PrevMain[u][k] == -1 || v < PrevMain[u][k]) {
                        PrevMain[u][k] = v;
                    }
                }
            }
        }
    }
}

// 5. Zielprüfung und Rekonstruktion der Hauptroute
long long min_total = INF;
int best_w_m = -1;

for(int u : works[m]) {
    if(MinArrival[u][m] == INF || dist_uv(u, T_id) == INF) continue;

    long long t = MinArrival[u][m] + dist_uv(u, T_id);
    if(t <= L) {
        if(t < min_total) {
            min_total = t;
            best_w_m = u;
        } else if(t == min_total) {
            // Numerisches Tie-Breaking
            if(best_w_m == -1 || u < best_w_m) {
                best_w_m = u;
            }
        }
    }
}
}

```

```

if(best_w_m == -1) {
    cout << "UNMOEGLICH\n";
    return;
}

cout << "MOEGLICH\n";

vector<int> W(m+1);
W[m] = best_w_m;
for(int k = m; k >= 2; --k) {
    W[k-1] = PrevMain[W[k]][k];
}

vector<long long> arr(m+1);
arr[1] = dist_uv(S_id, W[1]);
for(int k = 2; k <= m; ++k) {
    arr[k] = arr[k-1] + dist_uv(W[k-1], W[k]);
}

long long max_possible = min_total;
for(int k = 1; k <= m; ++k) {
    long long alt = arr[k] + BestAltDist[W[k]][k];
    if(alt > max_possible) {
        max_possible = alt;
    }
}

cout << max_possible << "\n\n";

auto print_path = [&](int u, int v, int step_for_v) {
    int curr = u;
    while (curr != v) {
        curr = next_node[curr][v];
        if (curr == v) {
            if (step_for_v != -1) {
                cout << " [" << node_names[v] << " " << step_for_v << "]\n";
            } else {
                cout << " " << node_names[v];
            }
        } else {
            cout << " " << node_names[curr];
        }
    }
};

cout << min_total << "\n";
cout << "S";
int curr = S_id;
for (int k = 1; k <= m; ++k) {
    print_path(curr, W[k], k);
    curr = W[k];
}
print_path(curr, T_id, -1);
cout << "\n";

for (int k = 1; k <= m; ++k) {
    long long alt_duration = arr[k] + BestAltDist[W[k]][k];
    cout << alt_duration << "\n";

    cout << node_names[W[k]];
    curr = W[k];
}

```

```
int v = NextAlt[W[k]][k];
print_path(curr, v, k);
curr = v;

for (int j = k + 1; j <= m; ++j) {
    int nxt = NextNormal[curr][j - 1];
    print_path(curr, nxt, j);
    curr = nxt;
}

print_path(curr, T_id, -1);
cout << "\n";
}

cout << "\n";
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    string fileN = "00";
    cerr << "Welches Beispiel? ";
    cin >> fileN;

    string in_dir = "inputs/" + fileN + ".txt";
    string out_dir = "outputs/" + fileN + "_out.txt";

    freopen(in_dir.c_str(), "r", stdin);

    freopen(out_dir.c_str(), "w", stdout);

    auto t0 = chrono::high_resolution_clock::now();
    solve();
    auto t1 = chrono::high_resolution_clock::now();
    chrono::duration<double> elapsed = t1 - t0;

    cerr << elapsed.count() << "s\n";

    return 0;
}
```


8 Literaturverzeichnis

1. **Dijkstra, E. W. (1959).** A Note on Two Problems in Connexion with Graphs. **Numerische Mathematik**, 1(1), 269–271.
2. **Bellman, R. (1957).** Dynamic Programming. Princeton University Press.
3. **Puterman, M. L. (2014).** Markov Decision Processes: Discrete Stochastic Dynamic Programming. John Wiley & Sons.